Project Overall

This project uses a real-world dataset (likely from a UK-based non-store online retail) to understand the financial health and operational patterns of the business. You are moving from Data Preparation (cleaning "dirty" data like negative quantities and null values) to Strategic Analysis (identifying who buys what, when, and where).

Initializing the environment by importing pandas and numpy, and loading the raw retail dataset to inspect the first few rows.

In [1]:
```
import pandas as pd
import numpy as np
```

In [2]:
```
df = pd.read_csv('Online_Retail.csv')
df.head(5)
```

Out[2]:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	Custom
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	12/1/10 8:26	2.55	178
1	536365	71053	WHITE METAL LANTERN	6	12/1/10 8:26	3.39	178
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	12/1/10 8:26	2.75	178
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	12/1/10 8:26	3.39	178
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	12/1/10 8:26	3.39	178

Generating descriptive statistics for all columns to identify data ranges, potential outliers (like negative quantities), and missing values.

In [5]:
```
df.describe(include ="all")
```

Out[5]:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	Un
count	541909	541909	540455	541909.000000	541909	541909.
unique	25900	4070	4223	NaN	23260	
top	573585	85123A	WHITE HANGING HEART T-LIGHT HOLDER	NaN	10/31/11 14:41	
freq	1114	2313	2369	NaN	1114	
mean	NaN	NaN	NaN	9.552250	NaN	4.
std	NaN	NaN	NaN	218.081158	NaN	96.
min	NaN	NaN	NaN	-80995.000000	NaN	-11062.
25%	NaN	NaN	NaN	1.000000	NaN	1.
50%	NaN	NaN	NaN	3.000000	NaN	2.
75%	NaN	NaN	NaN	10.000000	NaN	4.
max	NaN	NaN	NaN	80995.000000	NaN	38970.

Identifying missing data; specifically highlights significant gaps in CustomerID and Description

In [11]:
```
df.isnull().sum()
```

Out[11]:
```
InvoiceNo           0
StockCode           0
Description      1454
Quantity            0
InvoiceDate         0
UnitPrice           0
CustomerID     135080
Country             0
dtype: int64
```

Examining unique values and the total row count of the Quantity column to understand the scale of the data.

In [6]:
```
df.Quantity.unique()
```

```
Out[6]: array([     6,      8,      2,     32,      3,      4,     24,     12,
                   48,     18,     20,     36,     80,     64,     10,    120,
                   96,     23,      5,      1,     -1,     50,     40,    100,
                  192,    432,    144,    288,    -12,    -24,     16,      9,
                  128,     25,     30,     28,      7,     56,     72,    200,
                  600,    480,     -6,     14,     -2,     11,     33,     13,
                   -4,     -5,     -7,     -3,     70,    252,     60,    216,
                  384,    -10,     27,     15,     22,     19,     17,     21,
                   34,     47,    108,     52,  -9360,    -38,     75,    270,
                   42,    240,     90,    320,   1824,    204,     69,    -36,
                 -192,   -144,    160,   2880,   1400,     39,    -48,    -50,
                   26,   1440,     31,     82,     78,     97,     98,     35,
                   57,    -20,    110,    -22,    -30,    -70,   -130,    -80,
                 -120,    -40,    -25,    -14,    -15,    -69,   -140,   -320,
                   -8,    720,    156,    324,     38,     37,     49,     95,
                   -9,    -11,     29,     41,    -72,    -35,    -21,    -43,
                  -19,    -18,    -44,    402,    378,    150,    300,     54,
                  104,     67,    258,     66,     44,     55,     46,     99,
                   61,    408,    972,    208,   1008,   1000,    -77,   1488,
                  250,   1394,    400,   -223,   -150,    -13,    -33,   -723,
                 -177,     79,     84,    -32,   -100,    -28,    272,   -145,
                  -47,    -96,    113,     45,    106,     68,    267,    115,
                   65,   1728,    -60,    -16,     53,   -240,     76,    460,
                   71,     43,    213,     58,    576,   2400,    500,    180,
                 -300,   -500,    -23,    752,    960,   1296,    210,    172,
                  215,    129,    138,    116,    135,    197,   -106,    -54,
                  -17,   -939,    147,    168,    256,   -201,    -53,    -29,
                -2600,   -990,   -290,    -45,    860,   1010,   1356,   1284,
                  186,    114,    360,   1930,   2000,   3114,   1300,    670,
                  111,    211,     59,   -310,    -61,    -41,    176,    648,
                   62,  74215, -74215,    -64,    -84,     89,  -1400,     73,
                  -57,    112,    456,    -59,    -31,   5568,   2560,    136,
                  900,   -600,    -42,    -94,   -207,    -52,    130,   -206,
                 2592,    420,    800,    101,   1200,    864,   -217,     94,
                -1430,   1287,   -162,   -230,   -173,   -390,   -234,    504,
                  123,    118,    -76,   -200,   1056,   1500,    280,    407,
                  141,    124,    -99,     51,    -92,   -741,   3906,   -400,
                 -114,    102,   1152,    -88,    198,    117,     86,   -720,
                  125,    -86,   -391,    -87,   -278,    140,    228,   -154,
                -3000,     81,   -675,   -210,   -345,   -975,  -1200,  -1121,
                  -27,   -541,  -1277,   -197,   3000,  -1100,    -63,  -5368,
                  219,   -259,    232,    -82,   -178,    170,   -110,    768,
                   88,    700,   -102,   2160,   -323,   -450,   -232,    -83,
                 -153,   -524,  -2472,    -49,   -105,    342,    304,    167,
                  -34,    640,    175,    220,     74,     93,    164,    163,
                   63,    637,    122,    158,    165,    350,    -46,    -75,
                 -690,    -39,    -66,     83,    312,   -180,    392,   -624,
                 -194,    -26,    -62,    -37,    -91,   -139,   -158,  -1479,
                  -55,   -576,   -750,    330,    151,    -93,   -432,    -58,
                -1092,   -670,  -1300,  -1930,  -2000,  -3114,    462,    -65,
                  -68,    492,   -620,    273,    -90,   -170,  -1512,    -51,
                   85,    -56,   -160,   -360,    105,   -960,  -2376,   1350,
                  428,  -1350,    336,   -786,   -590,   -168,   -101,    -71,
                  132,   -413,   -664,    227,   2700,    222,    246,    906,
```

```
  756,    888,    552,    -97,    224,    738,    608,   -212,
 4300,    146,    143,    276,   -125,   -116,   -108,   4000,
 -304,   -272,  -1206,    -95,   1600,    323,   -161,   -472,
 -618,   -204,   1515,  -1515,  -9058,  -9600,    660,   -420,
 -126,   -220,   -271,  -1440,    264,    188,    588,    612,
  152,   -324,   -480,   -164,    -78,   -118,    430,  -1681,
   87,    155,    701,    828,    540,    696,    560,   -250,
 -408,   -179,    121,   -124,    512,   -251,  -3100,   3100,
 -169,    126,    291,   3186,  -2834,   -109,    109,   -121,
 -530,   -227,    261,   -346,    352,    142,    107,   -188,
-1060,   -342,   -288,    348,   1900,    157,   -343,   -455,
  425,    968,    684,    824,   -828,   -701,    196,    248,
  410,    236,    230,   -156,   -553,    145,    448,    245,
 -252,   -334,   -318,   -113,   -115,    171,   -242,    840,
 -967,   -203,  -3167,   -443,  -1897,    225,   -434,    750,
 -682,   -484,    682,    344,   -635,   -117,  -3667,    450,
  310,    494,   -384,     92,   1788,   -138,    624,    744,
  416,    496,    396,    306,   1878,   1944,    666,    708,
 1428,    852,   1412,    528,   -756,   -752,   -152,    -85,
 -312,    -79,   -147,    -67,   -131,    183,   -209,   -186,
 -231,   -129,   -458,   -275,  -2880,   1540,    672,   -800,
 -430,   -380,    -74,   -840,  -1296,   -365,   -104,   -270,
  -73,   -306,     91,    255,    468,   -468,   -111,   -184,
 -103,   -335,   4800,   -112,  -1000,    912,   1992,    184,
  148,   -657,  -1671,  -1158,  -2618,  -2003,   -674,  -4830,
 -905,  -1128,    832,    992,    630,   1020,   2100,    162,
-1560,  -1284,    -81,    314,    370,    131,    133,    484,
  149,    153,    257,    139,    137,    628,    179,   1820,
  478,    335,    253,    242,    375,   -276,   -256,    281,
  193,    181,    404,    244,    207,    199,   1130,    326,
  654,    688,    268,    249,   -864,    234,  -1510,   -550,
 -244,   -132,   -327,   -313,    279,   -398,   -280,    374,
 -504,   -696,   -149,   -224,   -428,    212,   -267,   -175,
12540,    760,    -98,    774,   -151,   2040,  -1131,   -135,
 1120,     77,   -900,    177,    490,    388,    620,   1404,
 -155,   -355,   -337,  -1050,   -338,    205,   -235,    698,
80995, -80995], dtype=int64)
```

In [8]:
```
df.Quantity.shape
```

Out[8]:
```
(541909,)
```

Creating a flag for negative quantities (likely returns or adjustments) and counting them to assess data "noise.

In [15]:
```
df['negetive_Quantity']= df['Quantity']<1
df.head()
```

Out[15]:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	Custom
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	12/1/10 8:26	2.55	178
1	536365	71053	WHITE METAL LANTERN	6	12/1/10 8:26	3.39	178
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	12/1/10 8:26	2.75	178
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	12/1/10 8:26	3.39	178
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	12/1/10 8:26	3.39	178

In [23]:
```
df['negetive_Quantity'].value_counts()
```

Out[23]:
```
negetive_Quantity
False    531285
True      10624
Name: count, dtype: int64
```

Removing the temporary flag column and verifying the dataframe structure.

In [24]:
```
df.drop(columns=['negetive_Quantity'], inplace=True)
```

In [26]:
```
df.head()
```

Out[26]:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	Custom
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	12/1/10 8:26	2.55	178
1	536365	71053	WHITE METAL LANTERN	6	12/1/10 8:26	3.39	178
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	12/1/10 8:26	2.75	178
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	12/1/10 8:26	3.39	178
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	12/1/10 8:26	3.39	178

Filtering the dataset to include only transactions with a Quantity > 1 to focus on actual sales and reduce bulk return bias.

In [31]:
```
df = df[df['Quantity'] >1]
```

In [32]:
```
df.Quantity.unique()
```

Out[32]:
```
array([    6,     8,     2,    32,     3,     4,    24,    12,    48,
          18,    20,    36,    80,    64,    10,   120,    96,    23,
           5,    50,    40,   100,   192,   432,   144,   288,    16,
           9,   128,    25,    30,    28,     7,    56,    72,   200,
         600,   480,    14,    11,    33,    13,    70,   252,    60,
         216,   384,    27,    15,    22,    19,    17,    21,    34,
          47,   108,    52,    75,   270,    42,   240,    90,   320,
        1824,   204,    69,   160,  2880,  1400,    39,    26,  1440,
          31,    82,    78,    97,    98,    35,    57,   110,   720,
         156,   324,    38,    37,    49,    95,    29,    41,   402,
         378,   150,   300,    54,   104,    67,   258,    66,    44,
          55,    46,    99,    61,   408,   972,   208,  1008,  1000,
        1488,   250,  1394,   400,    79,    84,   272,   113,    45,
         106,    68,   267,   115,    65,  1728,    53,    76,   460,
          71,    43,   213,    58,   576,  2400,   500,   180,   752,
         960,  1296,   210,   172,   215,   129,   138,   116,   135,
         197,   147,   168,   256,   860,  1010,  1356,  1284,   186,
         114,   360,  1930,  2000,  3114,  1300,   670,   111,   211,
          59,   176,   648,    62, 74215,    89,    73,   112,   456,
        5568,  2560,   136,   900,   130,  2592,   420,   800,   101,
        1200,   864,    94,  1287,   504,   123,   118,  1056,  1500,
         280,   407,   141,   124,    51,  3906,   102,  1152,   198,
         117,    86,   125,   140,   228,    81,  3000,   219,   232,
         170,   768,    88,   700,  2160,   342,   304,   167,   640,
         175,   220,    74,    93,   164,   163,    63,   637,   122,
         158,   165,   350,    83,   312,   392,   330,   151,   462,
         492,   273,    85,   105,  1350,   428,   336,   132,   227,
        2700,   222,   246,   906,   756,   888,   552,   224,   738,
         608,  4300,   146,   143,   276,  4000,  1600,   323,  1515,
         660,   264,   188,   588,   612,   152,   430,    87,   155,
         701,   828,   540,   696,   560,   121,   512,  3100,   126,
         291,  3186,   109,   261,   352,   142,   107,   348,  1900,
         157,   425,   968,   684,   824,   196,   248,   410,   236,
         230,   145,   448,   245,   171,   840,   225,   750,   682,
         344,   450,   310,   494,    92,  1788,   624,   744,   416,
         496,   396,   306,  1878,  1944,   666,   708,  1428,   852,
        1412,   528,   183,  1540,   672,    91,   255,   468,  4800,
         912,  1992,   184,   148,   832,   992,   630,  1020,  2100,
         162,   314,   370,   131,   133,   484,   149,   153,   257,
         139,   137,   628,   179,  1820,   478,   335,   253,   242,
         375,   281,   193,   181,   404,   244,   207,   199,  1130,
         326,   654,   688,   268,   249,   234,   279,   374,   212,
       12540,   760,   774,  2040,  1120,    77,   177,   490,   388,
         620,  1404,   205,   698, 80995], dtype=int64)
```

In [33]:
```
df.Quantity.shape
```

Out[33]:
```
(383058,)
```

In [34]:
```
df.columns
```

Out[34]:
```
Index(['InvoiceNo', 'StockCode', 'Description', 'Quantity', 'InvoiceDate',
       'UnitPrice', 'CustomerID', 'Country'],
      dtype='object')
```

Ensuring all UnitPrice values are non-negative and verifying the resulting dataset size.

In [35]:
```
df.UnitPrice.unique()
```

Out[35]:
```
array([2.550e+00, 3.390e+00, 2.750e+00, 7.650e+00, 4.250e+00, 1.850e+00,
       1.690e+00, 2.100e+00, 3.750e+00, 1.650e+00, 4.950e+00, 9.950e+00,
       5.950e+00, 7.950e+00, 8.500e-01, 6.500e-01, 1.250e+00, 2.950e+00,
       1.950e+00, 4.200e-01, 1.800e+01, 1.060e+00, 6.950e+00, 1.095e+01,
       3.450e+00, 5.500e-01, 3.800e-01, 1.450e+00, 3.950e+00, 6.750e+00,
       1.495e+01, 1.695e+01, 6.450e+00, 1.275e+01, 8.500e+00, 3.820e+00,
       3.370e+00, 5.450e+00, 6.350e+00, 7.200e-01, 6.400e-01, 1.000e-01,
       3.200e-01, 1.480e+00, 3.575e+01, 4.650e+00, 2.900e-01, 3.350e+00,
       9.500e-01, 2.100e-01, 1.200e-01, 0.000e+00, 2.400e-01, 1.490e+00,
       1.550e+00, 3.250e+00, 2.250e+00, 8.950e+00, 2.510e+00, 1.660e+00,
       4.300e-01, 2.980e+00, 5.910e+00, 3.360e+00, 7.620e+00, 1.698e+01,
       1.102e+01, 8.470e+00, 4.210e+00, 8.100e-01, 1.280e+00, 5.060e+00,
       1.272e+01, 6.770e+00, 9.320e+00, 8.400e-01, 1.017e+01, 1.187e+01,
       1.995e+01, 4.600e+00, 1.050e+00, 1.357e+01, 1.613e+01, 2.549e+01,
       2.123e+01, 2.038e+01, 2.820e+00, 1.270e+00, 3.190e+00, 9.900e-01,
       2.130e+00, 5.090e+00, 1.900e-01, 1.075e+01, 7.500e-01, 3.600e-01,
       3.400e-01, 1.600e-01, 1.880e+00, 1.800e-01, 2.995e+01, 4.000e+01,
       1.443e+01, 1.840e+00, 2.110e+00, 3.830e+00, 2.800e+01, 4.295e+01,
       5.550e+00, 5.490e+00, 4.240e+00, 9.340e+00, 1.260e+00, 1.065e+01,
       3.000e-01, 8.300e-01, 5.940e+00, 6.600e+00, 1.930e+00, 3.210e+00,
       5.300e-01, 7.000e-02, 6.370e+00, 3.810e+00, 3.400e+01, 5.740e+00,
       2.979e+01, 3.995e+01, 4.780e+00, 1.984e+01, 2.310e+00, 6.380e+00,
       9.890e+00, 3.860e+00, 5.277e+01, 3.880e+00, 3.269e+01, 5.510e+00,
       2.540e+00, 3.490e+00, 1.400e-01, 2.195e+01, 1.450e+02, 1.994e+01,
       2.500e-01, 6.790e+00, 2.960e+00, 4.810e+00, 1.750e+02, 1.700e-01,
       2.500e+00, 7.500e+00, 6.250e+00, 2.120e+00, 1.250e+02, 5.000e-01,
       2.000e+00, 1.530e+00, 9.400e-01, 3.495e+01, 3.000e+00, 1.500e+01,
       8.070e+00, 1.000e+00, 3.500e+00, 4.000e-01, 3.500e+01, 1.595e+01,
       2.000e-01, 7.250e+00, 7.640e+00, 3.980e+00, 3.700e-01, 8.490e+00,
       6.500e+01, 2.495e+01, 3.240e+00, 2.170e+00, 1.013e+01, 2.400e+00,
       2.650e+00, 2.210e+00, 2.160e+00, 7.300e-01, 1.240e+00, 3.440e+00,
       4.680e+00, 2.220e+00, 2.690e+00, 4.620e+00, 6.910e+00, 9.160e+00,
       8.870e+00, 7.130e+00, 4.130e+00, 2.460e+00, 6.630e+00, 7.460e+00,
       5.790e+00, 3.290e+00, 1.663e+01, 8.290e+00, 4.960e+00, 1.630e+00,
       1.246e+01, 9.130e+00, 2.080e+00, 2.496e+01, 7.450e+00, 3.120e+00,
       2.920e+00, 7.900e-01, 2.900e+00, 2.070e+00, 1.079e+01, 9.700e-01,
       1.413e+01, 1.163e+01, 9.960e+00, 1.579e+01, 1.000e+01, 3.730e+00,
       9.150e+00, 1.996e+01, 1.100e+02, 1.040e+00, 4.980e+00, 7.900e+00,
       3.329e+01, 2.760e+00, 4.150e+00, 1.670e+00, 1.460e+00, 1.248e+01,
       5.380e+00, 1.800e+00, 1.329e+01, 7.070e+00, 2.480e+00, 2.079e+01,
       4.996e+01, 4.710e+00, 8.200e-01, 1.380e+00, 5.750e+00, 4.770e+00,
       2.917e+01, 2.200e-01, 5.630e+00, 6.000e-02, 3.900e-01, 4.120e+00,
       1.750e+00, 8.150e+00, 3.330e+00, 5.000e+00, 3.500e-01, 1.230e+00,
       5.830e+00, 2.756e+02, 6.980e+00, 4.580e+00, 2.890e+00, 8.250e+00,
       5.995e+01, 3.295e+01, 4.000e+00, 8.000e-02, 3.400e+00, 8.320e+00,
       1.496e+01, 3.595e+01, 6.300e-01, 6.800e-01, 2.320e+00, 1.700e+00,
       6.000e+00, 1.298e+01, 3.850e+00, 8.330e+00, 2.490e+00, 1.295e+01,
       1.500e+00, 2.450e+00, 1.790e+00, 9.860e+00, 1.195e+01, 3.140e+00,
       5.000e+01, 6.650e+00, 1.250e+01, 1.667e+01, 1.350e+00, 2.350e+00,
       7.050e+00, 3.300e+00, 1.740e+00, 3.150e+00, 6.620e+00, 1.115e+01,
       2.000e+01, 1.040e+01, 2.913e+01, 5.829e+01, 1.890e+00, 5.390e+00,
       6.900e-01, 7.400e-01, 2.660e+00, 2.330e+00, 6.780e+00, 5.880e+00,
       4.860e+00, 3.320e+00, 9.420e+00, 1.980e+00, 1.180e+00, 1.920e+00,
       6.420e+00, 4.270e+00, 1.680e+00, 5.355e+01, 4.175e+01, 1.900e+00,
```

```
       4.180e+00, 5.970e+00, 5.900e-01, 4.000e-02, 6.495e+02, 4.800e+00,
       3.910e+00, 3.900e+00, 6.400e+00, 8.000e+00, 2.800e-01, 6.200e-01,
       2.060e+00, 2.700e-01, 8.600e-01, 2.260e+00, 2.620e+00, 5.800e-01,
       9.800e-01, 7.850e+00, 6.500e+00, 6.890e+00, 5.320e+00, 2.290e+00,
       5.400e+00, 8.500e+01, 4.995e+01, 2.390e+00, 3.200e+00, 6.230e+00,
       4.300e+00, 1.020e+00, 1.080e+00, 1.420e+00, 3.280e+00, 1.430e+00,
       4.020e+00, 4.080e+00, 5.140e+00, 2.600e+00, 5.080e+00, 5.150e+00,
       1.039e+01, 2.880e+00, 7.080e+00, 4.990e+00, 5.795e+01, 5.100e+00,
       2.410e+00, 5.180e+00, 8.120e+00, 1.640e+00, 5.640e+00, 6.720e+00,
       1.960e+00, 9.980e+00, 4.670e+00, 1.167e+01, 8.000e-01, 6.100e-01,
       3.100e-01, 1.829e+01, 6.360e+00, 3.260e+00, 1.120e+00, 1.970e+00,
       6.670e+00, 1.400e+00, 2.850e+00, 1.600e+00, 5.660e+00, 3.580e+00,
       5.350e+00, 9.000e-01, 3.570e+00, 3.890e+00, 3.970e+00, 5.680e+00,
       2.910e+00, 2.300e+00, 1.221e+01, 1.665e+01, 4.500e+00, 2.080e+01,
       1.788e+01, 3.920e+00, 4.140e+00, 2.500e+01, 4.163e+01, 1.455e+01,
       3.130e+00, 1.710e+00, 7.480e+00, 1.100e+00, 1.110e+00, 5.170e+00,
       2.670e+00, 9.200e-01, 2.940e+00, 1.296e+01, 9.570e+00, 1.895e+01,
       1.313e+01, 6.710e+00, 2.040e+00, 5.820e+00, 1.029e+01, 5.130e+00,
       1.190e+00, 5.200e-01, 7.840e+00, 2.420e+00, 1.010e+00, 3.160e+00,
       4.700e+00, 9.000e-02, 1.700e+01, 4.500e+01, 7.800e-01, 2.990e+00,
       1.860e+00, 1.335e+01, 4.760e+00])
```

In [37]:
```
df.UnitPrice.shape
```

Out[37]:
```
(383058,)
```

In [38]:
```
df = df[df['UnitPrice'] >= 0]
```

```
0.0
```

In [39]:
```
df.UnitPrice.shape
```

Out[39]:
```
(383058,)
```

In [40]:
```
df.describe()
```

Out[40]:

	Quantity	UnitPrice	CustomerID
count	383058.000000	383058.000000	324610.000000
mean	14.391434	2.782704	15199.208296
std	184.562297	3.414330	1715.276043
min	2.000000	0.000000	12346.000000
25%	3.000000	1.250000	13767.000000
50%	6.000000	1.790000	15051.000000
75%	12.000000	3.750000	16717.000000
max	80995.000000	649.500000	18287.000000

Converting InvoiceDate to a datetime object and extracting Year, Month, and Day for time-series analysis.

In [45]:
```
# 1. Convert the column to datetime objects
df['InvoiceDate'] = pd.to_datetime(df['InvoiceDate'])
```

In [46]:
```
df['Year'] = df['InvoiceDate'].dt.year
```

In [48]:
```
df['Month'] = df['InvoiceDate'].dt.month
```

In [49]:
```
df['Day'] = df['InvoiceDate'].dt.day
```

In [50]:
```
df.head(5)
```

Out[50]:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	Custom
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	2010-12-01 08:26:00	2.55	178
1	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00	3.39	178
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	2010-12-01 08:26:00	2.75	178
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	2010-12-01 08:26:00	3.39	178
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	2010-12-01 08:26:00	3.39	178

Creating a Revenue column ($Quantity \times UnitPrice$) to measure the financial impact of each transaction.

In [54]:
```
df['Revenue'] = df['Quantity'] * df['UnitPrice']
```

In [55]:
```
df.head(5)
```

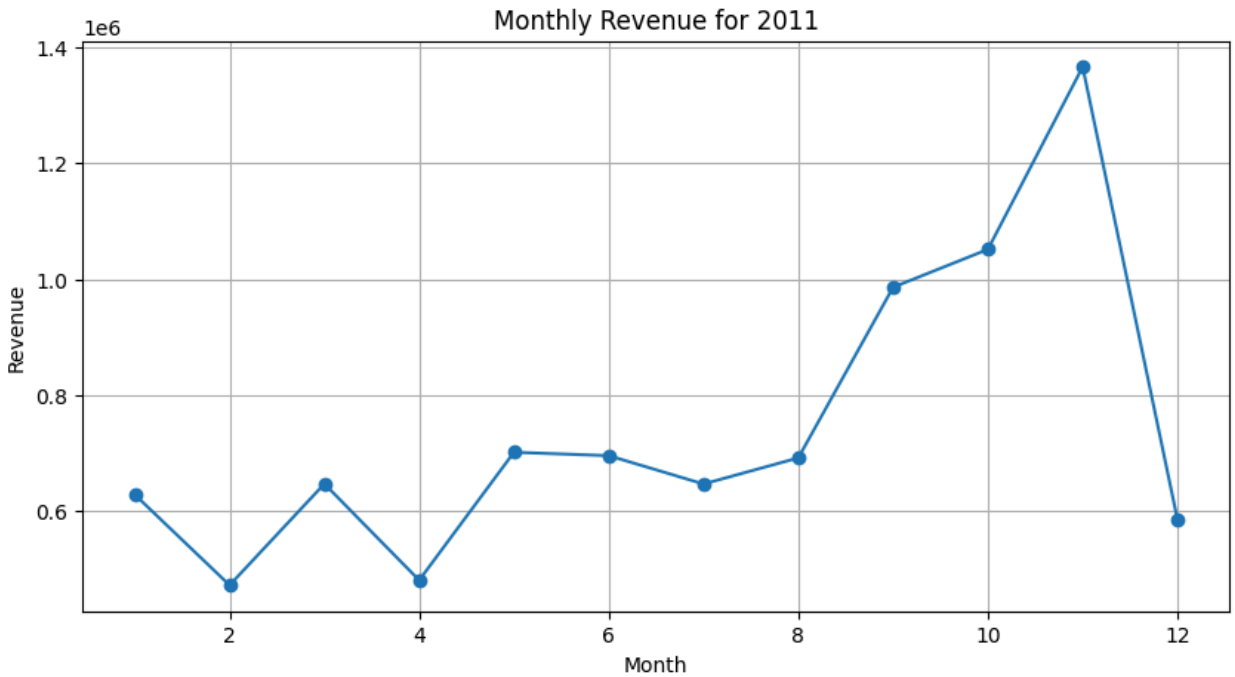
Out[55]:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	Custom
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	2010-12-01 08:26:00	2.55	178
1	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00	3.39	178
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	2010-12-01 08:26:00	2.75	178
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	2010-12-01 08:26:00	3.39	178
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	2010-12-01 08:26:00	3.39	178

Plotting monthly revenue for 2011 to identify seasonal peaks and growth patterns.

In [56]:
```
import matplotlib.pyplot as plt
df_2011 = df[df['Year'] == 2011]
monthly_revenue = df_2011.groupby('Month')['Revenue'].sum()
monthly_revenue.plot(kind='line', marker='o', figsize=(10, 5))
plt.title('Monthly Revenue for 2011')
plt.xlabel('Month')
plt.ylabel('Revenue')
plt.grid(True)
```

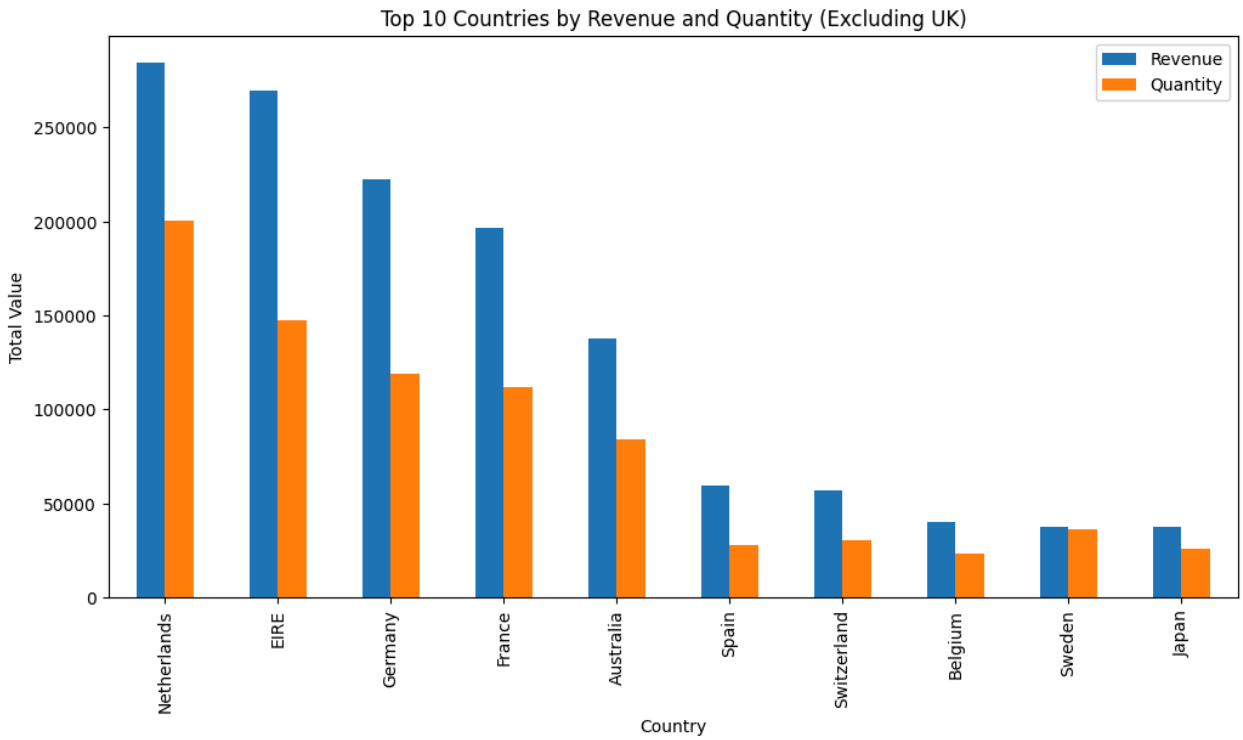
```
plt.show()
```



Insight: There is a clear upward trend in the latter half of the year. Revenue spikes significantly starting in September, peaking in November. This suggests strong seasonal demand likely driven by holiday shopping (Q4).

Visualizing the top 10 international markets (excluding the UK) by revenue and volume to identify key expansion zones

In [57]:
```
# Filter out United Kingdom
df_no_uk = df[df['Country'] != 'United Kingdom']
top_countries = df_no_uk.groupby('Country')[['Revenue', 'Quantity']].sum().sor
top_countries.plot(kind='bar', figsize=(12, 6))
plt.title('Top 10 Countries by Revenue and Quantity (Excluding UK)')
plt.ylabel('Total Value')
plt.show()
```

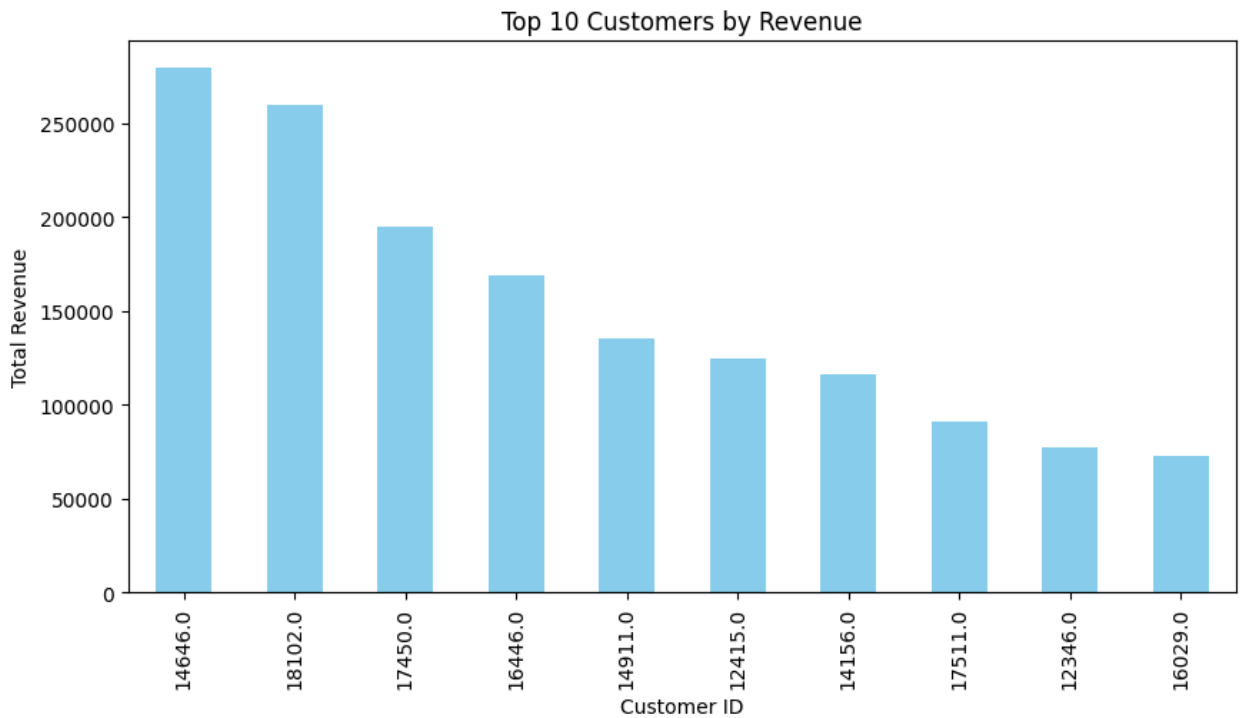


The Netherlands, EIRE (Ireland), and Germany are the leading international markets. The gap between the Netherlands and the others suggests a highly concentrated or wholesale-heavy customer base in that region.

Identifying the top 10 customers by total spend to highlight high-value accounts.

In [58]:
```
# Group by CustomerID and sum Revenue
top_customers = df.groupby('CustomerID')['Revenue'].sum().sort_values(ascendir

# Plotting
top_customers.plot(kind='bar', color='skyblue', figsize=(10, 5))
plt.title('Top 10 Customers by Revenue')
plt.xlabel('Customer ID')
plt.ylabel('Total Revenue')
plt.show()
```

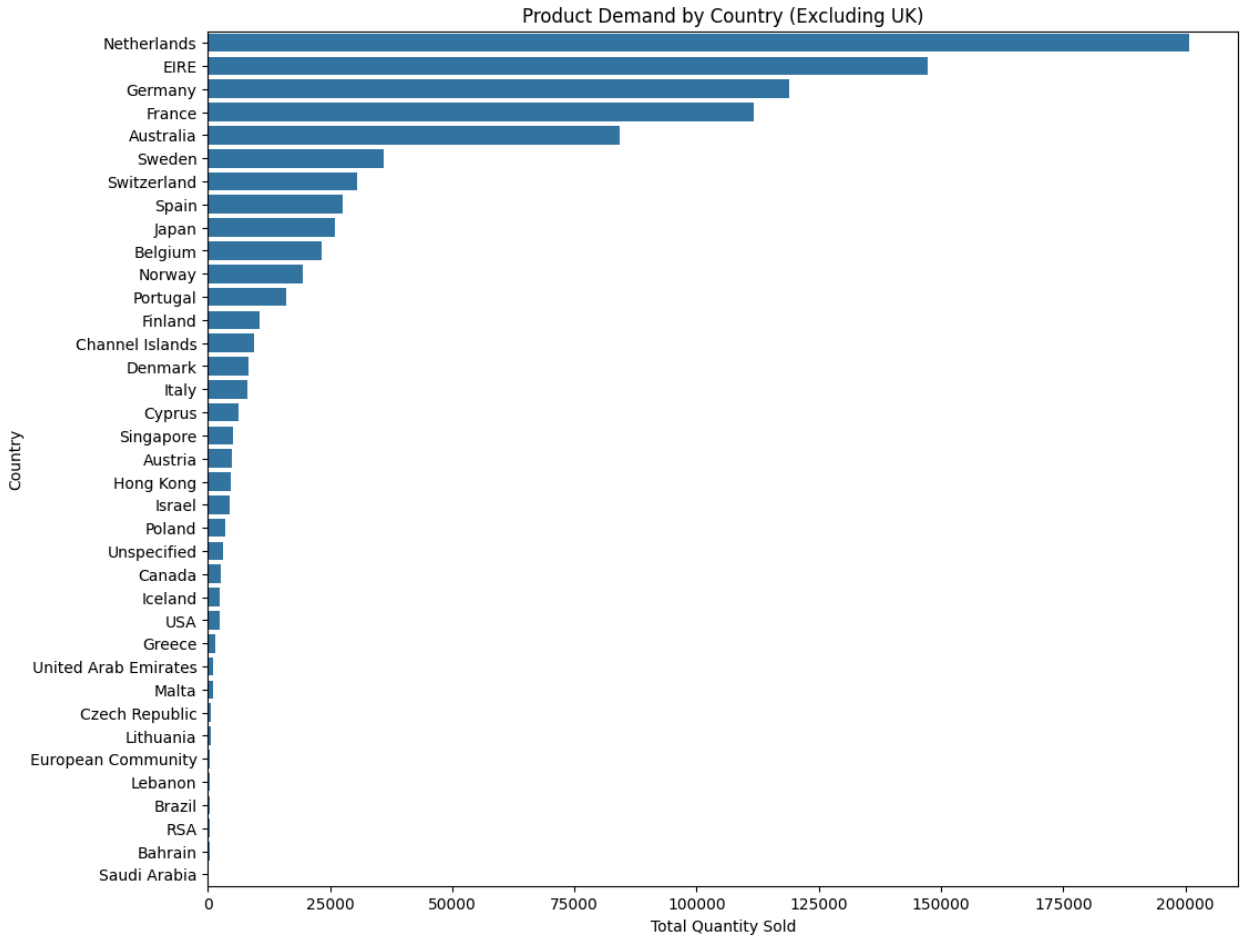


Insight: The revenue distribution among top customers shows a "long tail." A small group of Customer IDs contributes a disproportionately high amount of total revenue, indicating these are likely B2B or wholesale clients rather than individual retail shoppers.

In [59]:
```
import seaborn as sns

# Filter out UK and group by Country
demand_by_country = df[df['Country'] != 'United Kingdom'].groupby('Country')['

# Plotting all countries in one view
plt.figure(figsize=(12, 10))
sns.barplot(x=demand_by_country.values, y=demand_by_country.index)
plt.title('Product Demand by Country (Excluding UK)')
plt.xlabel('Total Quantity Sold')
plt.ylabel('Country')
plt.show()
```



Insight: Volume (Quantity) generally correlates with Revenue, but some countries show higher volume with lower revenue, suggesting they purchase lower-margin or cheaper items in bulk.

Saving the processed and cleaned dataset to Cleaned_Online_Retail.csv for further reporting or modeling.

In [62]:
```
df.to_csv('Cleaned_Online_Retail.csv', index=False)
```

Conclusion

The analysis reveals that while the United Kingdom remains the primary market, there is significant international traction in Northern Europe. The business experiences strong seasonality, with the final quarter of the year being critical for annual success. Furthermore, the presence of extreme outliers in quantity and high-value customers points toward a hybrid B2B/B2C business model.

Business Recommendations

Targeted Retention: Develop a "VIP Loyalty Program" specifically for the Top 10 Customer IDs identified in your analysis. Losing even one of these accounts would significantly impact total revenue.

Logistics Optimization: Since the Netherlands, Germany, and Ireland are top performers, consider localized warehousing or optimized shipping routes in these regions to reduce delivery times and costs.

Inventory Preparation: Given the Q4 revenue spike, inventory levels should be ramped up by late August. Focus stock on the "Top unique StockCodes" to avoid stockouts during the November peak.

Market Expansion: Explore why France and Australia (lower in the top 10) have lower demand compared to the Netherlands and implement targeted marketing campaigns in those regions to bridge the gap.

In [ ]: